Experiment 2.2

Experiment Title:-

Implementation of Merge Sort Using Divide and Conquer Technique for Recursive Data Sorting

Aim:-

To write a program to implement **Merge Sort using the Divide and Conquer technique** for recursively sorting a given set of data.

Topic Name:-

Merge Sort – Divide and Conquer Technique

Objective:-

1. To understand the Divide and Conquer approach.
2. To implement Merge Sort using recursion.
3. To divide an array into smaller subarrays.
4. To merge sorted subarrays into a single sorted array.
5. To analyze the time and space complexity of Merge Sort.

Software Requirements:- Visual Studio Code

Theory:-

Merge Sort:-

Merge Sort is a comparison-based sorting algorithm based on the Divide and Conquer technique. The algorithm works in three main steps:

- **Divide:** Divide the array into two approximately equal halves.
- **Conquer:** Recursively sort both halves.
- **Combine:** Merge the two sorted halves to form a single sorted array.

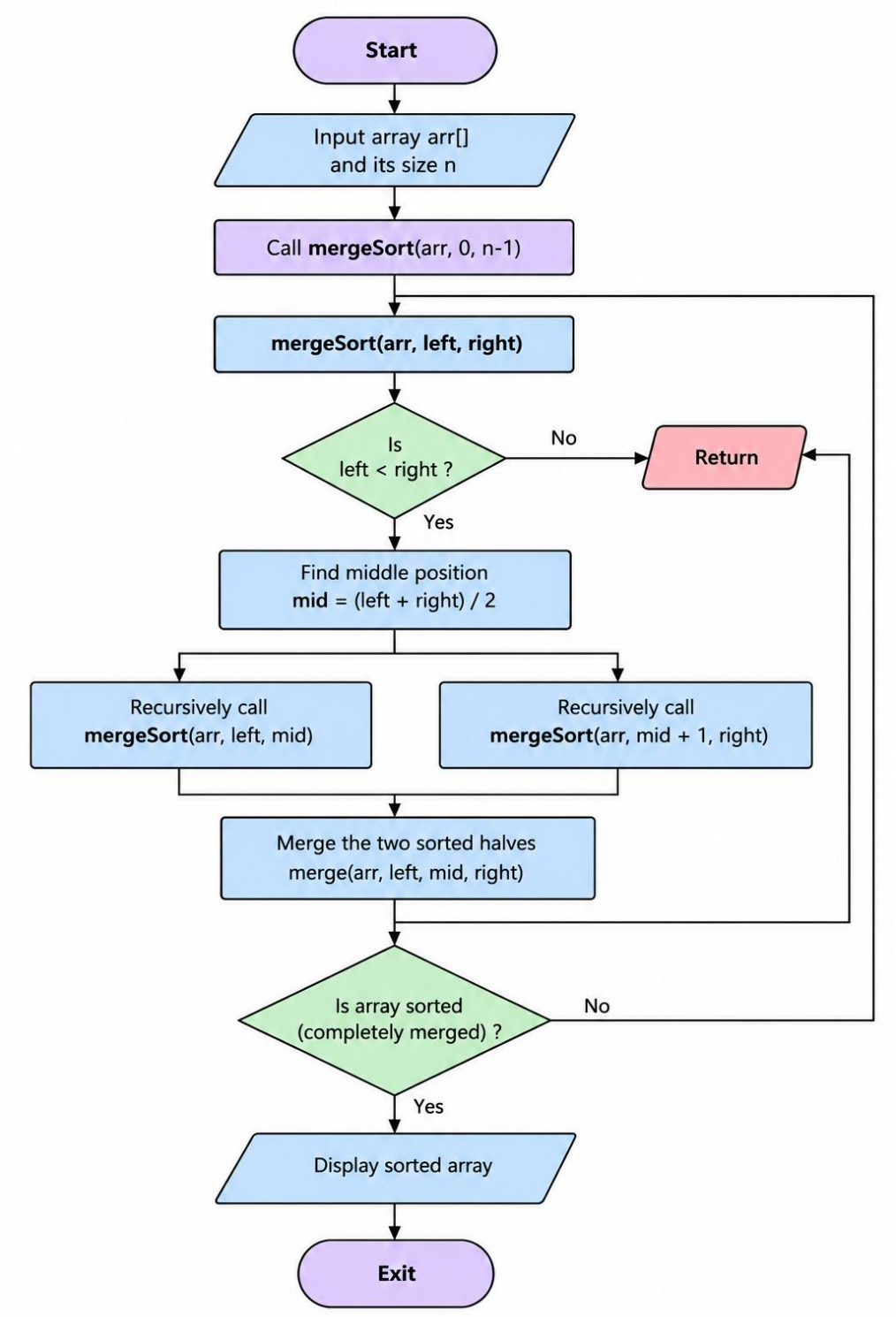
The process continues until each subarray contains only one element. A single element is already sorted, so the subarrays are then merged recursively.

Algorithm

Merge Sort Algorithm

1. Start.
2. Take an unsorted array.
3. Find the middle position of the array.
4. Divide the array into two halves.
5. Recursively apply Merge Sort to the left half.
6. Recursively apply Merge Sort to the right half.
7. Merge the two sorted halves.
8. Repeat until the complete array is sorted.
9. Display the sorted array.
10. Stop.

Flowchart:-


Start
Input array arr[]
and its size n
Call mergeSort(arr, 0, n-1)
mergeSort(arr, left, right)
Is
left < right ?
No
Return
Yes
Find middle position
mid = (left + right) / 2
Recursively call
mergeSort(arr, left, mid)
Recursively call
mergeSort(arr, mid + 1, right)
Merge the two sorted halves
merge(arr, left, mid, right)
Is array sorted
(completely merged) ?
No
Yes
Display sorted array
Exit

Implementation / Code:-

```
#include <iostream>
using namespace std;

void merge(int arr[], int left, int mid, int right) {
    int temp[100];
    int i = left;
    int j = mid + 1;
    int k = left;

    while (i <= mid && j <= right) {
        if (arr[i] < arr[j]) {
            temp[k] = arr[i];
            i++;
        } else {
            temp[k] = arr[j];
            j++;
        }
        k++;
    }

    while (i <= mid) {
        temp[k] = arr[i];
        i++;
        k++;
    }

    while (j <= right) {
        temp[k] = arr[j];
        j++;
        k++;
    }

    for (i = left; i <= right; i++) {
        arr[i] = temp[i];
    }
}

void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = (left + right) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        merge(arr, left, mid, right);
    }
}

int main() {
    int arr[] = {38, 27, 43, 3, 9, 82, 10};
    int n = 7;

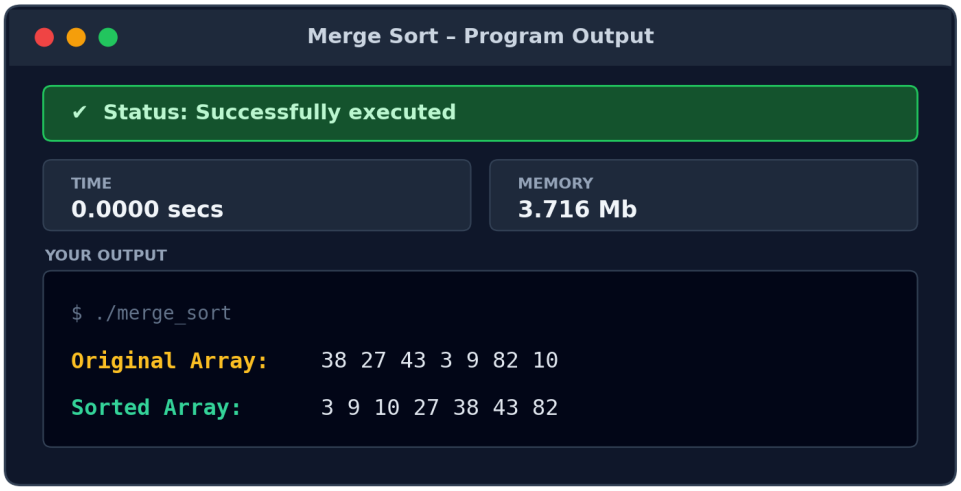
    cout << "Original Array: ";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";

    mergeSort(arr, 0, n - 1);

    cout << "\nSorted Array: ";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";

    return 0;
}
```

Output:-



Conclusion:-

Merge Sort was successfully implemented using the Divide and Conquer technique. The array was divided into smaller parts, each part was recursively sorted, and then the sorted parts were merged to obtain the final sorted array. Merge Sort provides efficient sorting with a time complexity of O(n log n).

Learning Outcomes:-

- Understand the Merge Sort algorithm.
- Understand the Divide and Conquer technique.
- Implement recursion in sorting.
- Divide an array into smaller parts.
- Merge sorted arrays.
- Analyze the time complexity of an algorithm.
- Sort data efficiently using O(n log n) time complexity.